

1

# PHP include file

# PHP Include File

2

- Insert the content of one PHP file into another PHP file before the server executes it
- Use the
  - `include()` generates a warning(E\_WARNING), but the script will continue execution
  - `require()` generates a fatal error (E\_ERROR), and the script will stop
- The `include` and `require` statements take all the text/code/markup that exists in a file, and inserts it into the file that uses the

# include() example

3

```
<?php  
echo '<a href="#">Home</a> - <a href="#">HTML Tutorial</a>  
- <a href="#">CSS Tutorial</a> - <a href="#">JavaScript  
Tutorial</a> - <a href="#">PHP Tutorial</a>';  
?>
```

```
<html>  
<body>  
  
<div class="leftmenu">  
<?php include("menu.php"); ?>  
</div>  
  
<h1>Welcome to my home page.</h1>  
<p>I have a great menu here.</p>  
  
</body>  
</html>
```



# include() example

4

[Home](#) - [HTML Tutorial](#) - [CSS Tutorial](#) - [JavaScript Tutorial](#) - [PHP Tutorial](#)

**Welcome to my home page!**

I have a great menu here.



# include() example

5

```
<?php  
$color = 'red';  
$car = 'BMW';  
?>  
vars.php
```

```
<html>  
<body>  
    <h1>Welcome to my home page!</h1>  
    <?php include("vars.php");  
    echo "I have a $color $car.";  
    ?>  
</body>  
</html>
```

*PHP*

# include() example

6

**Welcome to my home page!**

I have a red BMW.

# include() example

7

If a file is included with the include statement, and PHP cannot find it, it will send warnings.

```
<html>
<body>
    <h1>Welcome to my home page!</h1>
    <?php include("noFileExists.php");
    echo "I have a $color $car.";
    ?>
</body>
</html>
```

PHP



# include() example

8

**Welcome to my home page!**

**Warning:** include(noFileExists.php): Failed to open stream: No such file or directory in C:\xampp\htdocs\Back-end Discussion\sample.php on line 6

**Warning:** include(): Failed opening 'noFileExists.php' for inclusion (include\_path='C:\xampp\php\PEAR') in C:\xampp\htdocs\Back-end Discussion\sample.php on line 6

**Warning:** Undefined variable \$color in C:\xampp\htdocs\Back-end Discussion\sample.php on line 7

**Warning:** Undefined variable \$car in C:\xampp\htdocs\Back-end Discussion\sample.php on line 7

I have a .

# require() example

9

If a file is included with the require statement, and PHP cannot find it, it will send an error.

```
<html>
<body>
    <h1>Welcome to my home page!</h1>
    <?php require("noFileExists.php");
    echo "I have a $color $car.";
    ?>
</body>
</html>
```

PHP

# require() example

10

**Welcome to my home page!**

**Warning:** require(noFileExists.php): Failed to open stream: No such file or directory in **C:\xampp\htdocs\Back-end Discussion\sample.php** on line 6

**Fatal error:** Uncaught Error: Failed opening required 'noFileExists.php' (include\_path='C:\xampp\php\PEAR') in C:\xampp\htdocs\Back-end Discussion\sample.php:6 Stack trace: #0 {main} thrown in **C:\xampp\htdocs\Back-end Discussion\sample.php** on line 6



11

# PHP File Input/Output

# PHP file I/O functions

12

| function name(s)                                                                                                                                                                            | category                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <u>file</u> , <u>file_get_contents</u> ,<br><u>file_put_contents</u>                                                                                                                        | reading/writing entire files          |
| <u>basename</u> , <u>file_exists</u> ,<br><u>filesize</u> ,<br><u>fileperms</u> , <u>filemtime</u> , <u>is_dir</u> ,<br><u>is_readable</u> , <u>is_writable</u> ,<br><u>disk_free_space</u> | asking for information                |
| <u>copy</u> , <u>rename</u> , <u>unlink</u> ,<br><u>chmod</u> , <u>chgrp</u> , <u>chown</u> ,<br><u>mkdir</u> , <u>rmdir</u>                                                                | manipulating files and<br>directories |

# Reading/writing files

13

| contents of<br>foo.txt                   | file("foo.txt")                                                                                                    | file_get_contents("foo.txt")                        |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Hello<br>how are<br>you?<br><br>I'm fine | <pre>array(   "Hello\n",    #0   "how are\n",  #1   "you?\n",     #2   "\n",         #3   "I'm fine\n"  #4 )</pre> | <pre>"Hello\n how are\n you?\n \n I'm fine\n"</pre> |

- **file** returns lines of a file as an array
- **file\_get\_contents** returns entire contents of a file as a string



# Reading/writing file

14

Hello  
how are  
you?

I'm fine

*TXT*

```
<?php
    $txt = file("hello.txt");
    print_r($txt);
    echo "<br>";
    $txt2 = file_get_contents("hello.txt");
    print $txt2;
```

?>

*PHP*

# Reading/writing file

15

**Welcome to my home page!**

Array ( [0] => Hello [1] => how are [2] => you? [3] => [4] => I'm fine )  
Hello how are you? I'm fine

# Reading/writing an entire file

16

```
# reverse a file
$txt = file_get_contents("hello.txt");
$txt = strrev($txt);
file_put_contents("hello.txt", $txt);
echo file_get_contents("hello.txt");
```

*PHP*

- **file\_get\_contents** returns entire contents of a file as a string
- **file\_put\_contents** writes a string into a file, replacing any prior contents



# Reading/writing an entire file

17

**Welcome to my home page!**

enif m'I ?uoy era woh olleH

# Appending to a file

18

```
# add a line to a file
$new_text = "P.S. ILY, GTG TTYL!~";
file_put_contents("poem.txt", $new_text,
FILE_APPEND);
```

*PHP*

## old contents

Roses are red,  
Violets are blue.  
All my base,  
Are belong to you.

## new contents

Roses are red,  
Violets are blue.  
All my base,  
Are belong to you.  
P.S. ILY, GTG TTYL!~

# The file function

19

```
# display lines of file as a bulleted list
<?php
    $txts = file("hello.txt");
    foreach ($txts as $txt) { ?>
        <li><?= $txt ?></li>
    <?php }
?>
```

PHP

- file returns the lines of a file as an array of strings
  - each string ends with \n
  - to strip the \n off each line, use optional second

```
$lines = file("todolist.txt", FILE_IGNORE_NEW_LINES);
```

PHP



# Unpacking an array: `list`

20

```
list($var1, ..., $varN) = array;
```

*PHP*

```
$values = array("mundruid", "18", "f", "96");  
...  
list($username, $age, $gender, $iq) = $values;
```

*PHP*

- the `list` function accepts a comma-separated list of variable names as parameters
- use this to quickly "unpack" an array's contents into several variables

# Fixed-length files, file and list

21

```
Xenia Mountrouidou  
(919)685-2181  
570-86-7326  
personal.txt
```

*contents of file*

```
list($name, $phone, $ssn) = file("personal.txt");  
PHP
```

- reads the file into an array of lines and unpacks the lines into variables
- Need to know a file's exact length/format



# Splitting/joining strings

22

```
$array = explode(delimiter, string);  
$string = implode(delimiter, array);
```

*PHP*

```
$str = "CS 380 01";  
$arr = explode(" ", $str); # ("CS", "380", "01")  
$newStr = implode("...", $arr); # "CSE...380...01"
```

*PHP*

- `explode` and `implode` convert between strings and arrays



# Example explode

23

Harry Potter, J.K. Rowling  
The Lord of the Rings, J.R.R. Tolkien  
Dune, Frank Herbert

*contents of input file*

*books.txt*

```
<?php foreach (file("books.txt") as $book) {  
    list($title, $author) = explode(",", $book);  
    ?>  
    <p> Book title: <?= $title ?>, Author: <?= $author ?> </p>  
<?php  
}  
?>
```

*PHP*

# Example explode

24

Book title: Harry Potter, Author: J.K. Rowling

Book title: The Lord of the Rings, Author: J.R.R. Tolkien

Book title: Dune, Author: Frank Herbert

# Reading directories

25

| function       | description                                                                                                                |
|----------------|----------------------------------------------------------------------------------------------------------------------------|
| <u>scandir</u> | returns an array of all file names in a given directory (returns just the file names, such as "myfile.txt")                |
| <u>glob</u>    | returns an array of all file names that match a given pattern (returns a file path and name, such as "foo/bar/myfile.txt") |



# Example for glob

26

```
# reverse all poems in the poetry directory
$poems = glob("poetry/poem*.dat");
foreach ($poems as $poemfile) {
    $text = file_get_contents($poemfile);
    file_put_contents($poemfile, strrev($text));
    print "I just reversed " . basename($poemfile)
    . " ";
}
```

*PHP*

- ❑ glob can match a "wildcard" path with the \* character
- ❑ the basename function strips any leading directory from a file path

# Example for glob

27

```
# Suppose the directory contains:
```

```
poetry/
```

```
    poem1.dat
```

```
    poem2.dat
```

```
    poem3.dat
```

*Directories*

```
I just reversed poem1.dat I just reversed poem2.dat I just reversed poem3.dat
```

*output*

# Example for scandir

28

```
<ul>
<?php
$folder = "taxes/old";
foreach (scandir($folder) as $filename) {
    ?>
    <li> <?= $filename ?> </li>
<?php
}
?>
</ul>
```

PHP

- .
- ..
- 2009\_w2.pdf
- 2007\_1099.doc

*output*



29

# PHP Exceptions

# Exceptions

30

- Used to change the normal flow of the code execution if a specified error (exceptional) condition occurs.
- What normally happens when an exception is triggered:
  - current code state is saved
  - code execution will switch to a predefined (custom) exception handler function
  - the handler may then
    - resume the execution from the saved code state,
    - terminate the script execution or
    - continue the script from a different location in the

# Exceptions

31

- PHP uses the following keywords for exception handling:
  - **throw** - This keyword is used to throw exceptions.
  - **try** - This keyword is used to create a **try...catch**, or a **try...catch...finally** statement.
  - **catch** - This keyword is used to handle exceptions that are thrown by the code in a preceding **try** block.
  - **finally** - This keyword is used in **try...finally** and **try...catch...finally** to run a block of code whether or not an exception occurred.



# Exception throw example

32

```
<?php
//create function with an exception
function checkStr($str)
{
    if(strcmp($str, "correct") != 0)
    {
        throw new Exception("String is not correct!");
    }
    return true;
}

//trigger exception
checkStr("wrong");
?>
```

PHP

# Exception throw example

33

**Fatal error:** Uncaught Exception: String is not correct! in C:\xampp\htdocs\Back-end Discussion\sample.php:17 Stack trace: #0 C:\xampp\htdocs\Back-end Discussion\sample.php(23): checkStr('wrong') #1 {main} thrown in C:\xampp\htdocs\Back-end Discussion\sample.php on line 17

# Exception try...catch example

34

```
<?php
function divide($x, $y) {
    if($y == 0) {
        throw new Exception("Cannot divide by zero.");
    }
    return $x / $y;
}

try {
    echo divide(5, 0); // can throw an exception
} catch(Exception $e) {
    echo 'Error: ' . $e->getMessage();
}

echo '<br>Hello';
?>
```

PHP



# Exception try...catch example

35

Error: Cannot divide by zero.  
Hello

# Exception try...catch...finally example

36

```
<?php
function divide($x, $y) {
    if($y == 0) {
        throw new Exception("Cannot divide by zero.");
    }
    return $x / $y;
}

try {
    echo divide(5, 0); // can throw an exception
} catch(Exception $e) {
    echo 'Error: ' . $e->getMessage();
} finally {
    echo '<br>Process complete.';
}
?>
```

PHP

# Exception try...catch...finally example

37

Error: Cannot divide by zero.  
Process complete.



# Exception methods

38

- When catching an exception, the following table shows some of the methods that can be used to get information about the

| Method                               | Description                                                                               |
|--------------------------------------|-------------------------------------------------------------------------------------------|
| <a href="#"><u>getMessage()</u></a>  | Returns a string describing why the exception was thrown                                  |
| <a href="#"><u>getPrevious()</u></a> | If the exception was triggered by another one, this method returns the previous exception |
| <a href="#"><u>getCode()</u></a>     | Returns an integer which can be used to identify the exception                            |
| <a href="#"><u>getFile()</u></a>     | Returns the absolute path to the file where the exception occurred                        |

39

# PHP cookies and sessions

# Cookies

40

- Problem: HTTP is stateless
- What is a cookie?
  - tiny bits of information that a web site could store on the client's machine
  - they are sent back to the web site each time a new page is requested by this client.





# Bad Cookies?

41

- Urban myth: tracking, violate privacy
- Reality:
  - cookies are relatively harmless
  - can only store a small amount of information

# Sessions

42

- What is a session?
  - a combination of a server-side cookie and a client-side cookie,
  - the client-side cookie contains only a reference to the correct data on the server.
- when the user visits the site:
  - their browser sends the reference code to the server
  - the server loads the corresponding data.

# Cookies vs Sessions

43

- Cookies can be set to a long lifespan
- Cookies work smoothly when you have a cluster of web servers
- Sessions are stored on the server, i.e. clients do not have access to the information you store about
- Session data does not need to be transmitted with each page; clients just need to send an ID and the data is loaded from the local file.
- Sessions can be any size you want because



# Create a cookie

44

```
setcookie(name, value, expire, path, domain);
```

*PHP*

```
<?php  
setcookie("user", "Harry Potter", time()+3600);  
?>
```

```
<html>
```

```
.....
```

*PHP*

# Retrieve a Cookie Value

45

```
<?php
// Print a cookie
echo $_COOKIE["user"];

// A way to view all cookies
print_r($_COOKIE);
?>
```

*PHP*

# Delete a Cookie

46

```
<?php  
// set the expiration date to one hour ago  
setcookie("user", "", time()+3600);  
?>
```

*PHP*



# Session functions

47

- The most important session functions are:
  - `session_start()` - Starts a new session
  - `$_SESSION` - Stores and access session variables
  - `unset()` - Removes a specific session variable (e.g `unset($_SESSION["favcolor"])`)
  - `session_destroy()` - Destroys all data associated with the current session
  - `session_unset()` - Frees all session variables

# Start Session

48

```
<?php
// Start the session
session_start();
?>

<!DOCTYPE html>
<html>
<body>
<?php
// Set session variables
$_SESSION["favcolor"] = "green";
$_SESSION["favanimal"] = "cat";
echo "Session variables are set.";
?>
</body>
</html>
```



# Retrieve Session Variable Values

```
<?php
// Resume the session
session_start();
?>

<!DOCTYPE html>
<html>
<body>
<?php
// Output session variables that were set on previous page
if(isset($_SESSION["favcolor"])) {
    echo "Favorite color is " . $_SESSION["favcolor"] . "<br>";
    echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
} else {
    echo "No session data found.";
}
?>
</body>
</html>
```



# Modify Session Variable

```
<?php
// Resume the session
session_start();
?>
<!DOCTYPE html>
<html>
<body>

<?php
// to change a session variable, just overwrite it
$_SESSION["favcolor"] = "yellow";
print_r($_SESSION);
?>

</body>
</html>
```

# Destroy Session

```
<?php
// Resume the session
session_start();
?>

<!DOCTYPE html>
<html>
<body>
<?php
// Unset all session variables
session_unset();

// Destroy the session
session_destroy();

echo "You have been logged out."
?>
</body>
</html>
```



# PHP Sessions vs. Cookies

52

| Session                                                                             | Cookies                                              |
|-------------------------------------------------------------------------------------|------------------------------------------------------|
| The data is stored on the server                                                    | The data is stored in the browser                    |
| Expires when the browser is closed, or after a user has been inactive for some time | Can have a long expiration date                      |
| More secure, since data is not stored in the browser                                | Less secure, since data is stored in the browser     |
| More efficient, does not require constant data                                      | Less efficient, since data is sent with each request |